



UNIVERSITÀ DI PISA

DIPARTIMENTO DI MATEMATICA

Corso di Laurea Triennale in Matematica

Tensor Canonical Polyadic Decomposition and the Complexity of Matrix Multiplication

Tesi di Laurea Triennale

Relatore:

Prof. Leonardo Robol

Candidato:

Alberto Defendi

ANNO ACCADEMICO 2025/2026

SOMMARIO

La decomposizione CP (Canonical Polyadic Decomposition) di rango r di un tensore è la sua scomposizione nella somma di r tensori di rango uno, una generalizzazione naturale della decomposizione di matrici. Oltre ad avere un'importanza teorica nel calcolo tensoriale, è uno strumento utile in molte aree scientifiche: per citarne alcune, spettroscopia, *signal processing*, neurologia e analisi dei dati. In molte di queste applicazioni l'unicità della fattorizzazione è una proprietà fondamentale per rappresentare fedelmente le componenti di sistemi complessi—si pensi al problema di modellare i neuroni di un sistema nervoso tramite un tensore per studiarne gli impulsi, noto come *blind source separation*.

Il primo obiettivo è dimostrare l'unicità della fattorizzazione CP sotto opportune ipotesi. Questo risultato, originariamente dimostrato da Kruskal nel 1977 attraverso il concetto di Kruskal rank, è stato di recente semplificato seguendo la dimostrazione originale, usando strumenti geometrici. Inoltre, osserveremo alcune peculiarità teoriche che distinguono i tensori dalle matrici, ponendo le basi per l'utilizzo del calcolo tensoriale nella teoria della complessità.

Successivamente, applicheremo questi strumenti al problema della complessità della moltiplicazione tra matrici: in quanto mappa bilineare, il prodotto matriciale può essere rappresentato tramite un tensore tre indici, e poi messo in corrispondenza con un algoritmo per valutare il prodotto in funzione delle entrate del tensore. Introduciamo la nozione di complessità aritmetica del prodotto tra matrici per definire l'esponente asintotico ω , che ne misura la complessità ottimale. Tale quantità sarà messa in corrispondenza con il rango CP di un tensore tramite il Teorema di Strassen. Come corollario, ridurre lo studio della complessità ai tensori, anziché all'analisi diretta delle equazioni algebriche, semplifica notevolmente la ricerca di algoritmi più efficienti; una sfida aperta da decenni, dall'algoritmo di Strassen (del 1970), fino alle recenti scoperte guidate dall'intelligenza artificiale.

Infine, l'analisi verrà estesa dagli algoritmi di moltiplicazione esatti agli algoritmi approssimati (APA), introdotti da Bini et al. In questo contesto emerge il fenomeno topologico del border rank, per il quale una successione di tensori di rango inferiore può convergere a un tensore di rango superiore. Dal punto di vista algoritmico, ciò si traduce nella possibilità di approssimare la moltiplicazione esatta tramite una successione di algoritmi con costo computazionale inferiore. Di conseguenza, il rango tensoriale può essere sostituito con il border rank nel calcolo di ω , permettendo di ottenere stime asintotiche più accurate, al prezzo di un errore algoritmico arbitrariamente piccolo.

INDICE

1	APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA	1
1.1	Mappe multilineari e moltiplicazione tra matrici	1
1.2	L'esponente della moltiplicazione tra matrici	3
1.3	Decomposizione low-rank di forme bilineari	5
1.4	Rappresentazione del prodotto tra matrici	8
1.5	Il rango come misura della complessità	11
1.6	Algoritmi di moltiplicazione veloce	14
1.7	Alcune stime sull'esponente	18
1.8	Algoritmi APA	19
1.9	Errore algoritmico	21
2	APPENDICE	23
2.1	Il master method	23
	ACRONIMI	25
	GLOSSARIO	27
	BIBLIOGRAFIA	29

1 APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA

Gli *algoritmi di moltiplicazione veloce* sono degli algoritmi di moltiplicazione matriciale che impiegano un asintoticamente numero minore di operazioni aritmetiche rispetto all'algoritmo classico, ottenendo un esponente di complessità asintotica minore di 3 per le matrici quadrate.

Conoscere il rango CP di tensori che rappresentano applicazioni multilineari può rivelare informazioni sulla complessità asintotica e fornire algoritmi con complessità ottimale per la loro valutazione. Proviamo a formulare il problema.

descrivi cosa faremo

sistemare

1.1 MAPPE MULTILINEARI E MOLTIPLICAZIONE TRA MATRICI

Definizione 1.1.1 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , la moltiplicazione tra due matrici $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle: \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p} \\ (A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo “algoritmo”, ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Osservazione 1.1.2. La bilinearità della mappa sopra segue immediatamente dalla definizione del prodotto matriciale:

$$(1.1) \quad (AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj},$$

dove $i = 1, \dots, m$ e $j = 1, \dots, p$.

Controlla che non ci sia ambiguità nel riferirti a mappe/forme bilineari: $\psi: U \times V \rightarrow W$ è una forma bilineare se $W = \mathbb{K}$, altrimenti chiamala mappa bilineare

rinomina le forme bilineari con φ invece che f

Osservazione 1.1.3. Possiamo vedere la moltiplicazione tra matrici anche come una forma trilineare

$$\mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \times \mathbb{C}^{m \times p} \rightarrow \mathbb{C}$$

$$(A, B, C) \mapsto \text{trace}(ABC)$$

RAPPRESENTAZIONE IN COORDINATE DI UNA FORMA BILINEARE

forse spostata al
chap 1

Un modo semplice di rappresentare una forma bilineare $\varphi: U \times V \rightarrow W$ come un tensore è costruendone le slices. Fissato $k \in \{1, \dots, p\}$ e considerata la sua componente k -esima $\varphi_k: U \times V \rightarrow \mathbb{R}$, esiste una matrice $M \in \mathbb{R}^{m \times n}$ tale che $\varphi_k(u, v) = u^\top M v$ per ogni $(u, v) \in \mathbb{R}^m \times \mathbb{R}^n$. Scrivendo ogni componente di $\varphi = \begin{bmatrix} \varphi_1 \\ \vdots \\ \varphi_p \end{bmatrix}$ in questa forma, otteniamo un tensore $T \in \mathbb{R}^{m \times n \times p}$ tale che le sue fette frontali rappresentino ognuna delle p forme bilineari φ_k , ossia che

$$(1.2) \quad \varphi_k(u, v) = u^\top T_{:, :, k} v = \sum_{i=1}^m \sum_{j=1}^n T_{i,j,k} u_i v_j, \text{ per ogni } k = 1, \dots, p.$$

In forma tensoriale,

$$(1.3) \quad \varphi(u, v) = \begin{bmatrix} \varphi_1(u, v) \\ \vdots \\ \varphi_p(u, v) \end{bmatrix} = \begin{bmatrix} u^\top T_{:, :, 1} v \\ \vdots \\ u^\top T_{:, :, p} v \end{bmatrix} = T \times_1 u^\top \times_2 v^\top.$$

Osservazione 1.1.4. L'algebra multilineare garantisce l'esistenza e l'unicità di T .

Osservazione 1.1.5. Questa procedura si estende (passando alle coordinate) anche al caso in cui le entrate di u e v siano oggetti algebrici più complessi di scalari, ad esempio vettori o matrici.

Definizione 1.1.6 (Tensore strutturale). L'unico tensore $T \in U^* \otimes V^* \otimes W$ associato alla mappa bilineare $\varphi: U \times V \rightarrow W$ è detto *tensore strutturale* di φ .

ACHTUNG!
Può andare
bene?

Osservazione 1.1.7. Con un abuso di notazione tipicamente usato in letteratura [8, 15, 16], nel seguito identificheremo l'algoritmo di moltiplicazione matriciale (come mappa bilineare) con il suo tensore strutturale, denotando entrambi dello stesso simbolo $\langle m, n, p \rangle$, e specificheremo l'oggetto in questione laddove creerà ambiguità. Più avanti questa notazione sarà giustificata dalla Proposizione 1.3.3.

1.2 L'ESPONENTE DELLA MOLTIPLICAZIONE TRA MATRICI

In questo capitolo useremo il seguente modello computazionale per il calcolo della complessità, per calcolare costi computazionali anche su campi infiniti, e semplificare il calcolo dei costi eliminando branching (e.g. `if-then-else` o `while`).

Definizione 1.2.1 (Circuito aritmetico). Un *circuito aritmetico* Γ su $\mathbb{K}$ è un grafo diretto, aciclico, finito con vertici di grado entrante 0 o 2 ed esattamente un vertice di grado uscente 0. I vertici di grado entrante 0 sono etichettati con gli elementi di $\mathbb{K} \cup \{x_1, \dots, x_n\}$ e sono detti *inputs*. Quelli di grado entrante 2 sono etichettati con $+$ oppure $*$ e sono chiamati *gates*. Se il grado uscente di un vertice v è 0, allora v è chiamato un *output gate*. La dimensione di Γ è il numero di archi.

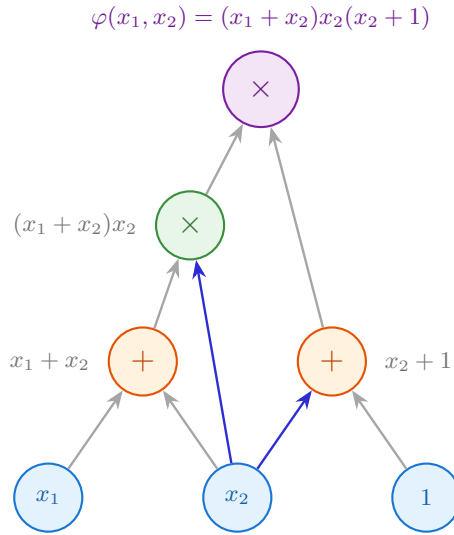


Figura 1.1: Esempio di circuito aritmetico per il polinomio $\varphi(x_1, x_2) = (x_1 + x_2)x_2(x_2 + 1)$.

Definizione 1.2.2 (Funzione costo). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo la *funzione costo* associata a un circuito aritmetico Γ come

$C_\Gamma(\varphi) :=$ numero di moltiplicazioni per calcolare φ sul circuito Γ , oppure

$C_\Gamma^{\text{tot}}(\varphi) :=$ numero di addizioni e moltiplicazioni per calcolare φ sul circuito Γ .

Denotiamo con $\mathcal{C}_\varphi$ come l'insieme dei circuiti aritmetici Γ che calcolano φ .

specifichiamo a che spazio appartengono i risultati intermedi?

Definizione 1.2.3 (Funzione di complessità aritmetica). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo

$$L(\varphi) := \inf_{\Gamma \in \mathcal{C}_\varphi} \{C_\Gamma(\varphi)\} \quad \text{la complessità moltiplicativa di } \varphi, \text{ e}$$

$$L^{\text{tot}}(\varphi) := \inf_{\Gamma \in \mathcal{C}_\varphi} \{C_\Gamma^{\text{tot}}(\varphi)\} \quad \text{la complessità totale di } \varphi.$$

Osservazione 1.2.4. In altre parole, $L(\varphi)$ è il minimo numero di moltiplicazioni non scalari sufficienti a calcolare φ , anche dette *active multiplications*. In maniera analoga, L^{tot} è il minimo numero di moltiplicazioni non scalari e di addizioni sufficienti a calcolare φ .

Consideriamo la forma bilineare $\varphi = \langle n, n, n \rangle$ data dall'Equazione 1.1. Fissato un campo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, e delle matrici $A, B \in \mathbb{R}^{n \times n}$, denotiamo con

$$(1.4) \quad M_{\mathbb{K}}(n) := L^{\text{tot}}(\langle n, n, n \rangle) = L^{\text{tot}}\left(\left\{\sum_{k=1}^n a_{ik} b_{kj} \mid i, j \in \{1, \dots, n\}\right\}\right)$$

il numero totale di moltiplicazioni aritmetiche sufficienti a calcolare la moltiplicazione di due matrici $n \times n$. Determinare una formula per $M_{\mathbb{K}}(n)$ è lontano dagli strumenti attualmente disponibili. Invece, il problema di approssimare la crescita asintotica della funzione $n \mapsto M_{\mathbb{K}}(n)$ è fattibile; definiamo

Definizione 1.2.5. L'esponente $\omega_{\mathbb{K}}$ di una moltiplicazione matriciale è il numero

$$\omega_{\mathbb{K}} := \inf_{n \in \mathbb{N}} \{\tau \in \mathbb{R} \mid \text{la moltiplicazione tra due matrici in } \mathbb{K}^{n \times n} \\ \text{ha costo } o(n^\tau) \text{ operazioni aritmetiche}\}.$$

In altre parole, $\omega_{\mathbb{K}} = \inf_{n \in \mathbb{N}} \{\tau \in \mathbb{R} \mid M_{\mathbb{K}}(n) = o(n^\tau)\}$.

Osservazione 1.2.6. Dalla notazione $M_{\mathbb{K}}(h)$ e $\omega_{\mathbb{K}}$ è dovuta alla dipendenza dal campo $\mathbb{K}$. Dato che tratteremo solo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, spesso scriveremo ω per $\omega_{\mathbb{K}}$.

Teorema 1.2.7 (Schönhage [19]). *L'esponente della moltiplicazione tra matrici è invariante rispetto alle estensioni scalari: se $\mathbb{K} \subset \mathbb{L}$ è un'estensione di campo, allora*

$$\omega_{\mathbb{K}} = \omega_{\mathbb{L}}$$

Osserviamo che $M_{\mathbb{K}}(n) \leq n^2(2n-1)$. Siccome il costo computazionale dev'essere almeno pari al numero di entrate della matrice, vale $M_{\mathbb{K}}(n) \geq n^2$, da cui $\omega \in [2, 3]$.

Trovare ω è un problema centrale nella teoria della complessità, e si congettura che $\omega = 2$, cioè che il costo della moltiplicazioni possa essere asintoticamente pari a quello delle moltiplicazioni. L'algoritmo naive ha $\omega = 3$, mentre il primo miglioramento, dovuto all'algoritmo di Strassen [22] mostra che $\omega \leq \log_2(7) < 2.81$. Il record attuale è $\omega \leq 2.371177$ [2]. Dopo Strassen, nel 1978 i miglioramenti significativi sono stati $\omega \leq 2.7962$ ad opera di [18], poi $\omega \leq 2.7799$ [4] nel 1979, poi $\omega \leq 2.55$ [19], poi $\omega \leq 2.48$ nel 1987 [21], e poi $\omega \leq 2.38$ [9] nel 1989, che potrebbe aver fatto pensare che nel 1990 si sarebbe trovato il bound. Dopo tale miglioramento, l'approssimazione è scesa molto lentamente, fino a scendere sotto $\omega < 2.372$ solo recentemente [1, 13, 20, 23]. Tuttavia, l'avvento dell'Intelligenza Artificiale sta dando un nuovo impulso a questa ricerca (si veda ad esempio AlphaTensor [12]); di recente (agosto 2026) l'utilizzo di strumenti di IA come AlphaEvolve ha stabilito il nuovo record di $\omega \leq 2.371177$ [2]. Sottolineiamo che da un punto di vista pratico, non è detto che le stime ottimali si traducano in algoritmi più veloci o stabili di quello classico, ad esempio l'algoritmo di Strassen, che presenteremo, risulta migliore di algoritmi con esponente più basso in molte situazioni [11].

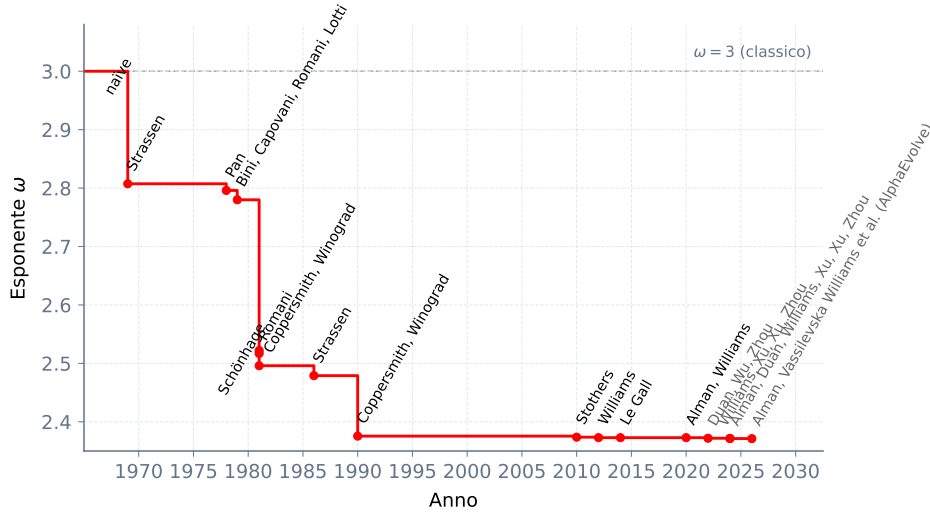


Figura 1.2: Miglioramenti del limite superiore dell'esponente della moltiplicazione matriciale ω dal 1969 ad oggi. La linea tratteggiata rappresenta il limite superiore $\omega \leq 3$.

1.3 DECOMPOSIZIONE LOW-RANK DI FORME BILINEARI

In § ?? abbiamo osservato che le fattorizzazioni low-rank riducono il costo in memoria. Aggiungiamo che questa rappresentazione è anche vantaggiosa per ridurre il

aggiungere
thm?

calcolo delle operazioni aritmetiche. Nel caso delle matrici, se $M \in \mathbb{R}^{m \times n}$ ha rango uno, possiamo scrivere $M = u \otimes v = uv^\top$ per certi $u \in \mathbb{R}^m$ e $v \in \mathbb{R}^n$; valutare $Mx = (uv^\top)x = u(v^\top x)$ ha costo $O(n)$, contro il costo $O(mn)$ del prodotto classico matrice vettore. In generale, se M ha rango R , possiamo scriverla come somma di R matrici di rango uno e per linearità ottenere un costo $O(Rn)$ per Mx . È possibile generalizzare quest'idea per mappe bilineari.

Calcolare la valutazione di una mappa bilineare φ come nell'Equazione 1.2 senza alcuna fattorizzazione di T ha un costo $O(mnp)$, data dal numero di moltiplicazioni necessarie fra gli elementi di u e v con le entrate non nulle di T . Osserveremo che queste operazioni si chiameranno *active multiplications*, in quanto saranno legate alla misura di complessità $L(\varphi)$. In particolare, sarà possibile ridurre le *active multiplications* trovando fattorizzazioni di rango più basso di T .

Definizione 1.3.1 (Bilinear algorithm). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali. Per ogni $i \in \{1, \dots, r\}$, siano $f_i \in U^*, g_i \in V^*, w_i \in W$ tali che

$$\varphi(u, v) = \sum_{i=1}^r f_i(u)g_i(v)w_i,$$

per ogni $u \in U, v \in V$. Allora la r -upla $(f_1, g_1, w_1; \dots; f_r, g_r, w_r)$ viene detta *bilinear computation (algorithm) di lunghezza r per φ* , e la lunghezza di più breve algoritmo di calcolo per φ è chiamata *bilinear complexity* o rango di φ , che indichiamo ancora con $\mathbf{R}(\varphi)$.

La prossima proposizione assicura che il rango di una forma bilineare è dato dal rango CP.

Proposizione 1.3.2. *Fissata una forma bilineare φ , e detto T il tensore che la rappresenta. Allora, dalla fattorizzazione CP di un tensore T si ottiene una bilinear computation, e in particolare $\mathbf{R}(\varphi) = \mathbf{R}(T)$.*

Dimostrazione. Data una decomposizione $T = \llbracket U, V, W \rrbracket$ di rango r , dalla Definizione ?? abbiamo che $T = \sum_{l=1}^r u_l \otimes v_l \otimes w_l$, o in componenti, $T_{i,j,k} = \sum_{l=1}^r u_{il}v_{jl}w_{kl}$. Inserendo questa relazione nell'Equazione 1.2 troviamo,

$$\begin{aligned}
 (1.5) \quad \varphi_k(x, y) &= \left(\left(\sum_{l=1}^r u_l \otimes v_l \otimes w_l \right) \times_1 x^\top \times_2 y^\top \right)_k = \left(\sum_{l=1}^r (u_l \otimes v_l \otimes w_l) \times_1 x^\top \times_2 y^\top \right)_k \\
 &= \sum_{i=1}^m \sum_{j=1}^n \sum_{l=1}^r u_{il} v_{jl} w_{kl} x_i y_j = \sum_{l=1}^r \left(\sum_{i=1}^m u_{il} x_i \right) \cdot \left(\sum_{j=1}^n v_{jl} y_j \right) w_{kl} \\
 &= \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_{kl},
 \end{aligned}$$

per ogni $k = 1, \dots, p$. Mettendo insieme le Equazioni 1.3 e 1.5 troviamo

$$(1.6) \quad \varphi(x, y) = \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_l.$$

Considerando i funzionali $f_l(x) := u_l^\top x$ e $g_l(y) := v_l^\top y$ indotti dal prodotto scalare standard si ha la tesi. □

Quindi valutare φ richiede esattamente $r = \mathbf{R}(T)$ active multiplications (e $O(r)$ addizioni), che sopra evidenziamo con $(\cdot)$. In particolare, il seguente risultato algebrico ci dice che la rappresentazione di una forma bilineare come tensore o come bilinear computation sono identiche, ed è utile per riscrivere un algoritmo tramite un tensore.

Proposizione 1.3.3 (Si veda [8], Cap. 14). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali. Allora esiste un unico isomorfismo $U^* \otimes V^* \otimes W \rightarrow \{\varphi : U \times V \rightarrow W, \text{ bilineare}\}$ che manda $f \otimes g \otimes w$ nella mappa bilineare $(u, v) \mapsto f(u)g(v)w$.*

La seguente proposizione lega la misura di complessità con il rango di una forma bilineare.

Proposizione 1.3.4 (Strassen, si veda [8], Capitolo 14).

Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali finiti, e $\varphi : U \times V \rightarrow W$ una mappa bilineare. Allora

$$L(\varphi) = \text{Lunghezza della più breve bilinear computation per } \varphi.$$

Osservazione 1.3.5. Da questa proposizione segue direttamente che

$$(1.7) \quad L(\varphi) \leq \mathbf{R}(\varphi) \leq 2L(\varphi).$$

in realtà è un corollario semplice di un teorema formulato da Strassen, che ho deciso di attribuire a Strassen (non aveva nome)

1.4 RAPPRESENTAZIONE DEL PRODOTTO TRA MATRICI

Torniamo al nostro focus iniziale, cioè valutare la forma bilineare del prodotto tra matrici con un numero ottimale di operazioni.

Consideriamo due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times p}$ e il loro prodotto $C \in \mathbb{R}^{m \times p}$. Ponendo $x = \text{vec}(A)$ e $y = \text{vec}(B)$, l'Equazione 1.3 diventa

$$(1.8) \quad \text{vec}(C^\top) = \varphi(x, y) = T \times_1 \text{vec}(A)^\top \times_2 \text{vec}(B)^\top,$$

dove tramite le trasposizioni abbiamo fissato l'ordinamento naturale sulle entrate di $\text{vec}(A)$ e $\text{vec}(B)$, e quello inverso su $\text{vec}(C^\top)$. Per costruzione, il tensore $T \in \mathbb{R}^{mn \times np \times mp}$ rappresenta l'algoritmo $\langle m, n, p \rangle$. Scrivendo T tramite la sua decomposizione CP di rango r , l'Equazione 1.6 ci permette di riscrivere l'espressione sopra come una bilinear computation

$$(1.9) \quad \text{vec}(C^\top) = \sum_{l=1}^r (u_l^\top \text{vec}(A)) \cdot (v_l^\top \text{vec}(B)) w_l = \sum_{l=1}^r (s_l \cdot t_l) w_l = \sum_{l=1}^r m_l w_l,$$

Dove abbiamo posto $s_l := s_l(A) := u_l^\top \text{vec}(A)$, $t_l := t_l(B) := v_l^\top \text{vec}(B)$ e $m_l := s_l t_l$.

Questa formula definisce un algoritmo di moltiplicazione veloce $\langle m, n, p \rangle$; spesso nella pratica m, n, p sono più piccoli delle matrici da moltiplicare; in questo caso è possibile dividerle a blocchi e applicare l'algoritmo in maniera ricorsiva. Nella prossima sezione giustificheremo meglio questa procedura.

TRATTARE IL CASO GENERALE

Date due matrici $n^i \times n^i$, possiamo vedere la loro moltiplicazione come il prodotto di matrici $n \times n$ sull'algebra delle matrici $n^{i-1} \times n^{i-1}$, suddividendole in blocchi. Questo dice anche che, se $\mathbf{R}(\langle n, n, n \rangle) \leq r$, allora $\mathbf{R}(\langle n^i, n^i, n^i \rangle) \leq r^i$; infatti, combinando le prossime Proposizioni 1.4.3 e 1.4.4, e usando che il rango è invariante per isomorfismo, troviamo

$$\mathbf{R}(\langle n^i, n^i, n^i \rangle) = \mathbf{R}(\langle n, n, n \rangle^{\otimes i}) \leq \mathbf{R}(\langle n, n, n \rangle)^i \leq r^i.$$

In altre parole, possiamo applicare ricorsivamente l'algoritmo di moltiplicazione veloce, che è disegnato per matrici $n \times n$, per ottenere la moltiplicazione di matrici $n^i \times n^i$ impiegando solo r^i moltiplicazioni (senza dover fattorizzare il tensore di $\langle n^i, n^i, n^i \rangle$, che sarebbe del modesto formato $(n^i)^2 \times (n^i)^2 \times (n^i)^2$).

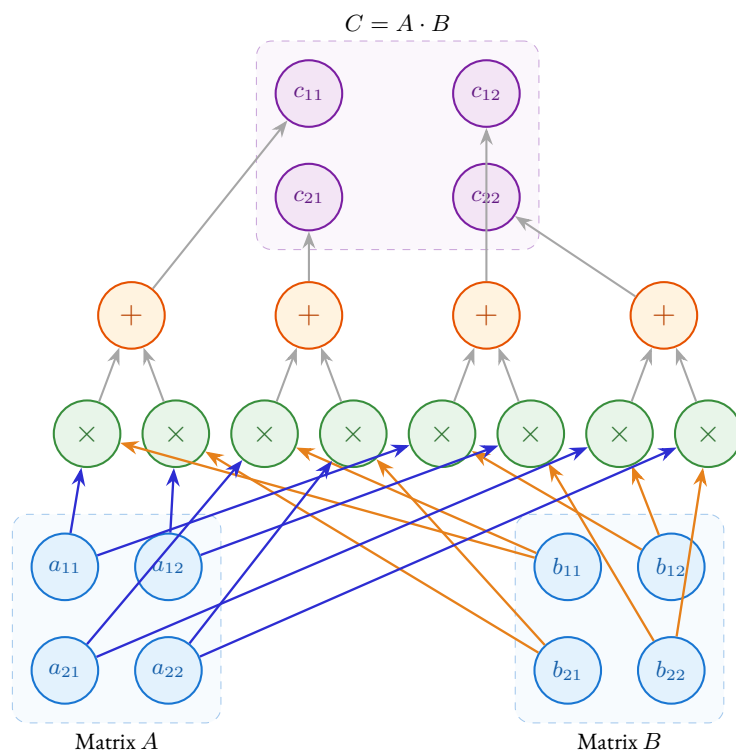


Figura 1.3: Circuito aritmetico per la moltiplicazione di matrici 2×2 .

Se invece $n > n_0$ sono due interi positivi (può essere utile pensare al caso $n_0 = 2$), dato un algoritmo con un caso base fissato $\langle n_0, n_0, n_0 \rangle$ è possibile ottenerne uno per $\langle n, n, n \rangle$ con $n > n_0$, allargando la matrice di taglia $n \times n$ aggiungendo zeri fino a raggiungere la dimensione $n_0^{\lceil \log_{n_0} n \rceil}$, in modo tale che le matrici da moltiplicare siano partizionate in blocchi di taglia divisibile dal caso base. La complessità dipenderà comunque da n_0 e da r , in quanto

$$\begin{aligned} R(\langle n, n, n \rangle) &\leq R(\langle n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil} \rangle) \quad \text{dalla 1.4.2,} \\ &\leq R(\langle n_0, n_0, n_0 \rangle)^{\lceil \log_{n_0} n \rceil} \quad \text{dalle 1.4.3 e 1.4.4,} \\ &\leq r^{\lceil \log_{n_0} n \rceil} \quad \text{per ipotesi } R(\langle n, n, n \rangle) \leq r, \\ &\leq r \cdot n^{\log_{n_0} r} \quad \text{usando che } \lceil x \rceil \leq x + 1. \end{aligned}$$

Riassumendo, $R(\langle n, n, n \rangle) \leq r \cdot n^{\log_{n_0} r}$. Dunque ignorando la costante e applicando l'Equazione 1.14, segue che $\omega \leq \log_{n_0} r$. Ad esempio, per l'algoritmo di Strassen (che introdurremo a breve) avremo che $r = 7, n_0 = 2$, da cui $\omega \leq \log_2 7 < 2.81$. Abbiamo dimostrato la seguente:

Proposizione 1.4.1. *Se $R(\langle n, n, n \rangle) \leq r$ per degli interi positivi n, r , allora $n^\omega \leq r$.*

Nei passaggi sopra abbiamo anche osservato che se $n \leq n'$ e dato un algoritmo per n' , possiamo ottenerne uno per n allargando la matrice. Possiamo racchiudere questa proprietà nel seguente risultato:

Proposizione 1.4.2 (Monotonia del rango di un algoritmo). *Il rango di un algoritmo è monotono crescente rispetto alla dimensione, ovvero dati due interi positivi $n \leq n'$, allora $R(\langle n, n, n \rangle) \leq R(\langle n', n', n' \rangle)$.*

Riportiamo le proposizioni utilizzate per dimostrare la proposizione sopra, dimostrate in [8, Capitoli 14 e 15].

Proposizione 1.4.3. *Siano e, e', h, h', l, l' degli interi positivi. Allora abbiamo*

$$\langle e, h, l \rangle \otimes \langle e', h', l' \rangle \simeq \langle ee', hh', ll' \rangle,$$

cioè le due forme bilineari sono isomorfe.

Proposizione 1.4.4. *Siamo φ_1, φ_2 due mappe bilineari. Allora $R(\varphi_1 \otimes \varphi_2) \leq R(\varphi_1)R(\varphi_2)$.*

da quello che ho capito quindi potremmo usare questa proposizione invece che il master theorem per calcolare la complessità?

forse si possono dimostrare con qualche conticino

1.5 IL RANGO COME MISURA DELLA COMPLESSITÀ

La prossima proposizione risponde al problema inverso: dato un intero positivo r , è possibile trovare un algoritmo con r moltiplicazioni?

Teorema 1.5.1 (Strassen [22], vedi anche [8] §15.1).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Ovvero, è possibile moltiplicare due matrici $n \times n$ usando $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se il rango tensoriale di $\langle n, n, n \rangle$ è $O(n^{\omega+\varepsilon})$.

Introduciamo alcuni strumenti utili a dimostrare questo risultato.

Definizione 1.5.2 (Mappa bilineare concise). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare. Diremo che φ è *concise* se valgono tutte e tre le condizioni sotto:

1. $\text{Ker}_L(\varphi) = \{u \in U \mid \varphi(u, \cdot) = 0\} = \{0\}$,
2. $\text{Ker}_R(\varphi) = \{v \in V \mid \varphi(\cdot, v) = 0\} = \{0\}$,
3. $\text{Im}(\varphi) = \{\varphi(u, v) \mid u \in U, v \in V\} = W$.

change to roman and to i -concise per matchare quella dei tensori

Osservazione 1.5.3.

Esempio 1.5.4. La mappa bilineare $\langle n, n, n \rangle$ è concise.

Lemma 1.5.5. Sia $x = (x_0, x_1, x_2, \dots)$ una successione di numeri reali che soddisfa la ricorsione $x_s = ax_{s-1} + cb^s$, per certi numeri reali a, b, c . Allora per ogni $s \geq 0$, vale

$$x_s = \begin{cases} a^s x_0 + (a^s - b^s) \frac{bc}{a-b} & \text{se } a \neq b, \\ (cs + x_0)a^s & \text{se } a = b. \end{cases}$$

Dimostrazione. Mostriamo la formula per induzione su s . Per $s = 0$ la formula è verificata.

$$\begin{aligned} x_1 &= ax_0 + bc, \\ x_2 &= a(ax_0 + bc) + b^2c \\ &= a^2x_0 + (a+b)bc \\ &= \begin{cases} a^2x_0 + (a^2 - b^2) \frac{bc}{a-b} & \text{se } a \neq b, \\ (2c + x_0)a^2 & \text{se } a = b. \end{cases} \end{aligned}$$

Assumiamo la proprietà vera per $n-1$ e mostriamola per n .

osservare che combacia con la def data per i tensori

Se $a \neq b$,

$$\begin{aligned} x_s &= a \left(a^{s-1} x_0 + (a^{s-1} - b^{s-1}) \frac{bc}{a-b} \right) + b^s c \\ &= a^s x_0 + ((a^{s-1} - b^{s-1})a + b^{s-1}(a-b)) \frac{bc}{a-b} \\ &= a^s x_0 + (a^s - b^s) \frac{bc}{a-b} \end{aligned}$$

Similmente, se $a = b$

$$\begin{aligned} x_s &= (c(s-1) + x_0 + c)a^s \\ &= (cs + x_0)a^s. \end{aligned}$$

□

Dimostrazione del Teorema di Strassen. Dimostriamo che

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Sia θ il lato destro dell'equazione sopra, e per semplicità scriviamo $M(n) := M_{\mathbb{K}}(n)$ e $\omega := \omega_{\mathbb{K}}$. Dall'Equazione 1.7 abbiamo

$$\frac{1}{2} \mathbf{R}(\langle n, n, n \rangle) \leq L(\langle n, n, n \rangle) \leq M(n),$$

e dalla Definizione 1.2.5 segue $\theta \leq \omega$.

Mostriamo che $\omega \leq \theta$. Per definizione di θ , per ogni $\varepsilon > 0$ esiste un $n > 1$ tale che

$$(1.10) \quad r := \mathbf{R}(\langle n, n, n \rangle) \leq n^{\theta+\varepsilon}.$$

matrix product,
ma TODO veri-
fica, e controlla
se è così davve-
ro(io ho i vec)

Dall'Equazione 1.9 esistono dei funzionali $f_\rho, g_\rho \in (\mathbb{K}^{n \times n})^*$ e delle matrici $w_\rho \in \mathbb{K}^{n \times n}$ per $\rho \in \{1, \dots, r\}$ tali che, per ogni $A, B \in \mathbb{K}^{n \times n}$

$$AB = \sum_{\rho=1}^r f_\rho(A) g_\rho(B) w_\rho.$$

Vedendo $\mathcal{A} = \mathbb{K}^{n^i \times n^i}$ come l'algebra delle matrici $n^i \times n^i$, notiamo che i funzionali $f_\rho, g_\rho : \mathbb{K}^{n \times n} \rightarrow \mathbb{K}$ si estendono a funzionali $f_\rho, g_\rho : \mathcal{A}^{n \times n} \rightarrow \mathcal{A}$, sostituendo le entrate scalari con delle matrici. In particolare vale, per ogni $A, B \in \mathcal{A}^{n \times n}$

$$(1.11) \quad AB = \sum_{\rho=1}^r f_\rho(A) g_\rho(B) w_\rho.$$

L'Equazione sopra rappresenta l'algoritmo di moltiplicazione a blocchi tra A e B : infatti abbiamo $\mathcal{A}^{n \times n} = (\mathbb{K}^{n^i \times n^i})^{n \times n} \simeq \mathbb{K}^{n^{i+1} \times n^{i+1}}$, dove l'isomorfismo può essere pensato come la suddivisione di matrici più grandi in matrici a blocchi. Da questo fatto, unito all'Equazione 1.11, si ottiene che

$$(1.12) \quad M(n^{i+1}) \leq rM(n^i) + cn^{2i},$$

dove $c := c(n, r)$ è una costante che dipende sia da n che da r . Questo perché nella formula il prodotto AB , sviluppato come prodotto tra blocchi $n^i \times n^i$ (formalmente, visto come prodotto a coefficienti in $\mathcal{A}$), coinvolge r prodotti e un numero costante di addizioni. Siccome $\langle n, n, n \rangle$ è concise, abbiamo che $r \geq n^2$. Fissando n (e quindi r), risolviamo la ricorsione dell'Equazione 1.12 in funzione di i usando il Lemma 1.5.5 con $a = r$ e $b = n^2$, analizzando due casi.

Se $r > n^2$, abbiamo che

$$\begin{aligned} M(n^i) &= r^i M(1) + (r^i - n^{2i}) \frac{n^2 c}{r - n^2} \\ &= (M(1) + n^2 c) r^i - n^{2i} \frac{n^2 c}{r - n^2}, \end{aligned}$$

da cui,

$$M(n^i) \leq \alpha r^i + \beta n^{2i},$$

dove abbiamo posto $\alpha := M(1) + n^2 c$ e $\beta := -\frac{n^2 c}{r - n^2}$.

Se $r = n^2$, dal Lemma 1.12 abbiamo

$$M(n^i) \leq (cs + M(1))r^i,$$

In entrambi i casi troviamo,

$$(1.13) \quad M(n^i) = O(r^i).$$

Dalla proposizione 1.4.2 segue che M è una successione monotona, per ogni $h \in \mathbb{N} \setminus \{0\}$,

$$M(h) \leq M(n^{\lceil \log_n h \rceil}) = O(r^{\log_n h}) = O(h^{\log_n r}) = O(h^{\theta + \varepsilon}),$$

dove la prima uguaglianza segue dall'Equazione 1.4.2, la seconda da una proprietà del logaritmo, e la terza applicando la stima dell'Equazione 1.10. Quindi passando all'inf, $\omega_{\mathbb{K}} \leq \theta$, che completa la dimostrazione. $\square$

non chiaro se anche l'equazione sopra garantisce monotonia

Osservazione 1.5.6. Riassumendo, il teorema sopra mette in corrispondenza $\mathbf{R}(\langle n, n, n \rangle)$, ovvero il rango del tensore che rappresenta $\langle n, n, n \rangle$ e numero di active multiplications, con l'esponente asintotico ottimale. Possiamo quindi scrivere

$$(1.14) \quad \omega = \lim_{n \rightarrow +\infty} \log_n(\mathbf{R}(\langle n, n, n \rangle)).$$

1.6 ALGORITMI DI MOLTIPLICAZIONE VELOCE

Osservazione 1.6.1. Di seguito ci limiteremo a studiare prodotti fra matrici quadrate; per il caso rettangolare, si possono estendere i ragionamenti sopra considerando una rappresentazione di rango r del tensore $\langle m, n, p \rangle$ e scrivendo, attraverso una serie di permutazioni cicliche, la fattorizzazione di $\langle mnp, mnp, mnp \rangle$ di rango r^3 , che produce un algoritmo di complessità $O(n^{\log_{abc} r^3}) = O(n^{3 \log_{abc} r})$. Questo si può vedere tramite la scrittura della moltiplicazione tra matrici come forma trilineare dell'Osservazione 1.1.3, che ci permette di dire che il tensore è invariante per trasformazioni cicliche, in quanto

$$(1.15) \quad \text{trace}(ABC) = \text{trace}(CAB) = \text{trace}(BAC).$$

IL CASO NAIVE

Dire se la derivazione del tensore è chiara, ho provato a giustificarla meglio che potessi

Trattiamo il caso di matrici quadrate con taglie potenze di due (possiamo farlo a meno di passare a una sottomatrice di taglia pari). In questo caso possiamo partizionare le matrici in delle sottomatrici a blocchi 2×2 ¹:

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}.$$

L'algoritmo naive procede combinando un insieme di otto moltiplicazioni matriciali con quattro addizioni:

$$(1.16) \quad \begin{aligned} m_1 &= a_{11} \cdot b_{11} = s_1 t_1 & m_2 &= a_{12} \cdot b_{21} = s_2 t_2 & m_3 &= a_{11} \cdot b_{12} = s_3 t_3 \\ m_4 &= a_{12} \cdot b_{22} = s_4 t_4 & m_5 &= a_{21} \cdot b_{11} = s_5 t_5 & m_6 &= a_{22} \cdot b_{21} = s_6 t_6 \\ m_7 &= a_{21} \cdot b_{12} = s_7 t_7 & m_8 &= a_{22} \cdot b_{22} = s_8 t_8 \end{aligned}$$

¹possiamo pensare alle entrate sia come scalari, che come matrici.

$$\begin{aligned} c_{11} &= m_1 + m_2 = a_{11}b_{11} + a_{12}b_{21}, & c_{12} &= m_3 + m_4 = a_{11}b_{12} + a_{12}b_{22}, \\ c_{21} &= m_5 + m_6 = a_{21}b_{11} + a_{22}b_{21}, & c_{22} &= m_7 + m_8 = a_{21}b_{12} + a_{22}b_{22}. \end{aligned}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^2 a_{ik}b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j}$$

 Figura 1.4: Rappresentazione della moltiplicazione di due matrici 2×2 .

Proviamo a scriverlo in forma tensoriale. La sua bilinear computation è

$$C = \sum_{i=1}^8 m_i w_i$$

Usando l'Osservazione 1.1.3 possiamo scrivere $\langle 2, 2, 2 \rangle$ come mappa trilineare:

$$\begin{aligned} \text{trace}(ABC) &= \text{trace} \left(\begin{bmatrix} s_1 t_1 + s_2 t_2 & s_3 t_3 + s_4 t_4 \\ s_5 t_5 + s_6 t_6 & s_7 t_7 + s_8 t_8 \end{bmatrix} \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \right) \\ &= (s_1 t_1 + s_2 t_2) c_{11} + (s_3 t_3 + s_4 t_4) c_{21} + (s_5 t_5 + s_6 t_6) c_{12} + (s_7 t_7 + s_8 t_8) c_{22}. \end{aligned}$$

Poniamo

$$\begin{aligned} w_1 &= w_2 = c_{11}, & w_5 &= w_6 = c_{12}, \\ w_3 &= w_4 = c_{21}, & w_7 &= w_8 = c_{22}. \end{aligned}$$

Tramite l'isomorfismo della Proposizione 1.3.3, possiamo mappare ogni termine

$s_r t_r w_r$ dell'equazione sopra in $s_r \otimes t_r \otimes c_r$ e scrivere il tensore strutturale di $\langle 2, 2, 2 \rangle$ in formato CP (si veda anche la Figura 1.5):

$$\begin{aligned}
 \langle 2, 2, 2 \rangle &= a_{11} \otimes b_{11} \otimes c_{11} + a_{12} \otimes b_{21} \otimes c_{11} \\
 &+ a_{21} \otimes b_{11} \otimes c_{21} + a_{22} \otimes b_{21} \otimes c_{21} \\
 &+ a_{11} \otimes b_{12} \otimes c_{12} + a_{12} \otimes b_{22} \otimes c_{12} \\
 &+ a_{21} \otimes b_{12} \otimes c_{22} + a_{22} \otimes b_{22} \otimes c_{22}.
 \end{aligned}
 \tag{1.17}$$

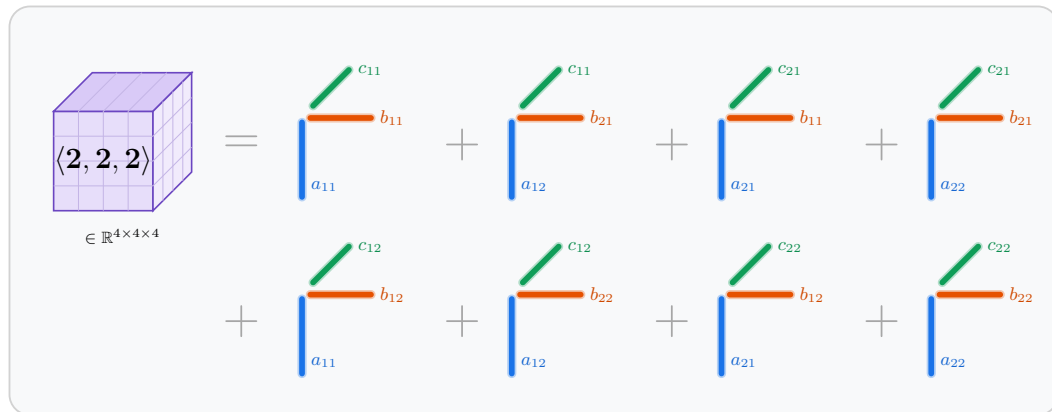


Figura 1.5: Decomposizione CP del tensore di moltiplicazione di matrici $\langle 2, 2, 2 \rangle$ come somma di 8 tensori di rango uno.

non mi convince molto

mst

Osserviamo che questo caso base dà luogo a un algoritmo ricorsivo per $n > 2$. Ripercorrendo il ragionamento dell'Equazione del Teorema di Strassen, troviamo che $M(n) = O(n^{\log_2 8}) = O(n^3)$. Aggiungiamo che, in letteratura a volte si usa un risultato chiamato Master Theorem [10] per dare queste stime asintotiche.

Per secoli questo algoritmo è stato ritenuto il più veloce per moltiplicare le matrici 2×2 (probabilmente anche l'unico conosciuto). Nel 1969 Strassen² tentò di mostrare che non esistesse un algoritmo più veloce di quello classico. Tramite una serie di riduzioni riuscì a mostrare l'ottimalità su $\mathbb{Z}/2\mathbb{Z}$, ma per $\mathbb{R}$ o $\mathbb{C}$ si rivelò essere un fallimento spettacolare.

L'ALGORITMO DI STRASSEN

L'algoritmo di Strassen è probabilmente l'algoritmo di moltiplicazione veloce più noto e segue la stessa struttura a blocchi $\langle 2, 2, 2 \rangle$.

²in una comunicazione riportata da Landsberg in [16].

$$\begin{array}{lll}
 s_1 = a_{11} + a_{22} & s_2 = a_{21} + a_{22} & s_3 = a_{11} \\
 s_4 = a_{22} & s_5 = a_{11} + a_{12} & s_6 = a_{21} - a_{11} \\
 s_7 = a_{12} - a_{22} & & \\
 t_1 = b_{11} + b_{22} & t_2 = b_{11} & t_3 = b_{12} - b_{22} \\
 t_4 = b_{21} - b_{11} & t_5 = b_{22} & t_6 = b_{11} + b_{12} \\
 t_7 = b_{21} + b_{22} & &
 \end{array}$$

$$\begin{aligned}
 (1.18) \quad & m_r = s_r t_r, \quad 1 \leq r \leq 7 \\
 & c_{11} = m_1 + m_4 - m_5 + m_7 \quad c_{12} = m_3 + m_5 \\
 & c_{21} = m_2 + m_4 \quad c_{22} = m_1 - m_2 + m_3 + m_6
 \end{aligned}$$

Dall'equazione sopra possiamo scrivere i termini w_r come combinazione lineare dei c_{ij} dati raccogliendo i termini m_r . In altre parole,

$$\begin{aligned}
 w_1 &= c_{11} + c_{22} \\
 w_2 &= c_{21} - c_{22} \\
 w_3 &= c_{12} + c_{22} \\
 w_4 &= c_{11} + c_{21} \\
 w_5 &= -c_{11} + c_{12} \\
 w_6 &= c_{22} \\
 w_7 &= c_{11}.
 \end{aligned}$$

Possiamo anche calcolare gli m_r :

$$\begin{aligned}
 m_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) \\
 m_2 &= (a_{21} + a_{22})b_{11} \\
 m_3 &= a_{11}(b_{12} - b_{22}) \\
 m_4 &= a_{22}(b_{21} - b_{11}) \\
 m_5 &= (a_{11} + a_{12})b_{22} \\
 m_6 &= (a_{21} - a_{11})(b_{11} + b_{12}) \\
 m_7 &= (a_{12} - a_{22})(b_{21} + b_{22})
 \end{aligned}$$

Mettendo insieme troviamo,

$$\begin{aligned} \sum_{i=1}^7 m_i w_i &= (a_{11} + a_{22})(b_{11} + b_{22})(c_{11} + c_{22}) \\ &\quad + (a_{21} + a_{22})b_{11}(c_{21} - c_{22}) \\ &\quad + a_{11}(b_{12} - b_{22})(c_{12} + c_{22}) \\ &\quad + a_{22}(-b_{11} + b_{21})(c_{11} + c_{21}) \\ &\quad + (a_{11} + a_{12})b_{22}(-c_{11} + c_{12}) \\ &\quad + (a_{21} - a_{11})(b_{11} + b_{12})c_{22} \\ &\quad + (a_{12} - a_{22})(b_{21} + b_{22})c_{11}. \end{aligned}$$

non chiaro passaggio con forma trilineare, cioè ottengo la bilinear computation con $\text{tr}(ABC^T)$

Che in forma tensoriale diventa

$$\begin{aligned} \langle 2, 2, 2 \rangle &= (a_{11} + a_{22}) \otimes (b_{11} + b_{22}) \otimes (c_{11} + c_{22}) \\ &\quad + (a_{21} + a_{22}) \otimes b_{11} \otimes (c_{21} - c_{22}) \\ &\quad + a_{11} \otimes (b_{12} - b_{22}) \otimes (c_{12} + c_{22}) \\ &\quad + a_{22} \otimes (-b_{11} + b_{21}) \otimes (c_{21} + c_{11}) \\ &\quad + (a_{11} + a_{12}) \otimes b_{22} \otimes (-c_{11} + c_{12}) \\ &\quad + (-a_{11} + a_{21}) \otimes (b_{11} + b_{12}) \otimes c_{22} \\ &\quad + (a_{12} - a_{22}) \otimes (b_{21} + b_{22}) \otimes c_{11}. \end{aligned}$$

aggiungere costo

Espandendo i prodotto tensoriali e facendo opportune semplificazioni, possiamo ritrovare il tensore dell'Equazione 1.17.

1.7 ALCUNE STIME SULL'ESPONENTE

Ad esempio, l'algoritmo di Strassen [22] mostra che $\mathbf{R}(\langle 2, 2, 2 \rangle) \leq 7$, e come dimostrato da Winograd [24], $\mathbf{R}(\langle 2, 2, 2 \rangle) = 7$, ovvero non è possibile moltiplicare due matrici 2×2 con meno di sette moltiplicazioni. Nei casi diversi da $\langle 2, 2, 2 \rangle$ il problema si complica, e attualmente abbiamo alcune stime più lasche: già per matrici 3×3 , sappiamo solo che $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$ [7, 14]; questo caso è ritenuto molto curioso perché esiste un'espressione nota per il tensore, ma è difficile dimostrarne il rango. Mentre la migliore stima asintotica dal basso per il caso di matrici $n \times n$ è $\frac{5}{2}n^2 - 3n \leq \mathbf{R}(\langle n, n, n \rangle)$ [6]. Si veda [16, Capitolo 11] per una discussione più ampia.

1.8 ALGORITMI APA

Il border rank
viene introdot-
to nel capitolo
precedente

Dopo il tentativo di Strassen di dimostrare che l'algoritmo di moltiplicazione standard fosse ottimale, Bini et. Al [4] considerano il seguente algoritmo di moltiplicazione ridotto ottenuto ponendo a zero l'entrata a_{22} di $\langle 2, 2, 2 \rangle$:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} \end{bmatrix}.$$

Facendo alcuni raccoglimenti e sostituendo $a_{22} = 0$, il tensore dell'Equazione 1.17 si riscrive come

$$\begin{aligned} \langle 2, 2, 2 \rangle^{\text{red}} &:= a_{11} \otimes (b_{11} \otimes c_{11} + b_{12} \otimes c_{12}) + a_{12} \otimes (b_{21} \otimes c_{11} + b_{22} \otimes c_{12}) \\ &\quad + a_{21} \otimes (b_{11} \otimes c_{21} + b_{12} \otimes c_{22}). \end{aligned}$$

Che è somma di sei tensori elementari distinti, quindi $\mathbf{R}\langle 2, 2, 2 \rangle^{\text{red}} \leq 6$. Bini et. al. provarono a cercare un'espressione di rango cinque per $\langle 2, 2, 2 \rangle^{\text{red}}$ numericamente, calcolando

$$\min_{\substack{T \in \mathbb{R}^{n \times n \times n} \\ \mathbf{R}(T)=5}} \|\langle 2, 2, 2 \rangle^{\text{red}} - T\|$$

norma di $\langle 2, 2, 2 \rangle^{\text{red}}$. Il loro computer continuava a produrre tensori T di rango cinque con la norma della differenza che diventava sempre più piccola, ma con coefficienti sempre più grandi. Questo è andato avanti per un po' di tempo, fino a che realizzarono che il codice era giusto, ma quello che sembrava un problema era in realtà una manifestazione del border rank. L'espressione del tensore che stavano cercando era il seguente tensore di rango 5:

$$\langle n, n, n \rangle_t^{\text{red}} = \frac{1}{t} [(I) + (II) - (III) - (IV) + (V)],$$

dove

qui z_1^2 con z_2^1
è scambiato
rispetto alla
source, control-
la di nuovo

$$\begin{aligned}
(I) &= (x_2^1 + tx_1^1) \otimes (y_2^1 + ty_2^2 \otimes z_2^1) \\
&= x_2^1 \otimes y_2^1 \otimes z_2^1 + t[x_2^1 \otimes y_2^2 \otimes z_2^1 + x_1^1 \otimes y_2^1 \otimes z_2^1] + O(t^2) \\
(II) &= (x_1^2 + tx_1^1) \otimes y_1^1 \otimes (z_1^1 + tz_1^2) \\
&= x_1^2 \otimes y_1^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^1 \otimes z_1^2 + x_1^1 \otimes y_1^1 \otimes z_1^1] + O(t^2) \\
(III) &= x_1^2 \otimes y_1^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^1 \otimes z_1^2 + x_1^1 \otimes y_1^1 \otimes z_1^1] + O(t^2) \\
(IV) &= x_1^2 \otimes ((y_1^1 + y_2^1) + ty_1^2) \otimes z_1^1 \\
&= x_1^2 \otimes y_1^1 \otimes z_1^1 + x_1^2 \otimes y_2^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^2 \otimes z_1^1] \\
(V) &= (x_2^1 + x_1^2) \otimes (y_2^1 + ty_1^2) \otimes (z_1^1 + tz_2^2) \\
&= x_2^1 \otimes y_2^1 \otimes z_1^1 + x_1^2 \otimes y_2^1 \otimes z_1^1 \\
&\quad + t[x_2^1 \otimes (y_2^1 + z_2^2 + y_1^2 \otimes z_1^1) + x_1^2 \otimes (y_2^1 \otimes z_2^2 + y_1^2 \otimes z_1^1)] + O(t^2)
\end{aligned}$$

In ogni passaggio abbiamo applicato più volte la proprietà distributiva del prodotto tensoriale. Osserviamo che i termini di grado costante in t si cancellano:

$$\begin{aligned}
&x_2^1 \otimes (y_2^1 \otimes z_2^1 - y_2^1 \otimes z_1^1 - y_2^1 \otimes z_2^1 + y_2^1 \otimes z_1^1) \\
&+ x_1^2 \otimes (y_1^1 \otimes z_1^1 - y_1^1 \otimes z_1^1 + y_2^1 \otimes z_1^1 - y_2^1 \otimes z_1^1) = 0
\end{aligned}$$

Sommando e raggruppando i termini di grado uno in t troviamo

$$\begin{aligned}
\langle n, n, n \rangle^{\text{red}} &= x_1^1 \otimes (y_1^1 \otimes z_1^1 + y_2^1 \otimes z_2^1) + \\
&\quad x_2^1 \otimes (y_1^2 \otimes z_1^1 + y_2^2 \otimes z_2^1) + \\
&\quad x_1^2 \otimes (y_1^1 \otimes z_1^2 + y_2^1 \otimes z_2^2),
\end{aligned}$$

ovvero il tensore di partenza. Da cui,

$$\begin{aligned}
\langle n, n, n \rangle_t^{\text{red}} - \langle n, n, n \rangle^{\text{red}} &= \frac{1}{t} \left(t \langle n, n, n \rangle^{\text{red}} + O(t^2) \right) - \langle n, n, n \rangle^{\text{red}} \\
&= O(t) \xrightarrow{t \rightarrow 0} 0
\end{aligned}$$

Abbiamo dimostrato che i tensori approssimanti $\langle n, n, n \rangle_t^{\text{red}}$, di rango 5 convergono ad un tensore di rango 6. Nella pratica, è possibile calcolare il prodotto di due matrici usando il tensore $\langle n, n, n \rangle_t^{\text{red}}$ per una scelta di t maggiore della precisione di macchina, riducendo il numero di moltiplicazioni (in questo caso di una moltiplicazione), a costo di un errore algoritmico che accenneremo in § 1.9.

Il border rank si rivela utilissimo nello studio dell'esponente ω , cioè possiamo sostituire il rango con il border rank nel Teorema 1.5.1:

Teorema 1.8.1 (Bini [4], vedi §3.2).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \underline{\mathbf{R}}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Anche se potremmo avere $\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$, non sono abbastanza diversi da avere un impatto sull'esponente.

why?

Riportiamo alcune stime sul border rank riportate in [15, §1]. Per $n = 2$, Winograd dimostra che $\underline{\mathbf{R}}(\langle 2, 2, 2 \rangle) = \mathbf{R}(\langle 2, 2, 2 \rangle) = 7$. Ci aspetteremmo che per $n > 2$, $\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$, ma la risposta non è così semplice. Per $n = 3$, sappiamo solo che $15 \leq \underline{\mathbf{R}}(\langle 3, 3, 3 \rangle) \leq 20$, mentre per il rango vale $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$. In generale, sappiamo che $\mathbf{R}(\langle n, n, n \rangle) \geq 3n^2 - o(n)$ e $\underline{\mathbf{R}}(\langle n, n, n \rangle) \geq 2n^2 - \lceil \log_2(n) \rceil - 1$.

ref

1.9 ERRORE ALGORITMICO

Discutiamo brevemente l'errore algoritmico degli algoritmi di moltiplicazione veloci, rimandando a [3] per ulteriori dettagli. Gli algoritmi di moltiplicazione veloci producono errori numerici più grandi rispetto all'algoritmo classico, dove l'errore in avanti è dato componente per componente, e dipende solo dal prodotto scalare della riga e colonna corrispondente. Gli algoritmi veloci, invece, eseguono calcoli che coinvolgono altre entrate della matrice che non appaiono nel prodotto scalare (eventualmente questi contributi si cancellano), dunque gli errori dipendono da proprietà globali delle matrici di input. In gergo, gli algoritmi veloci ³ sono detti *stabili rispetto alla norma* [5, 17], mentre quello classico gode di una proprietà più forte, ovvero è *stabile rispetto alle componenti*, caratteristica irraggiungibile per gli algoritmi veloci [17]. Definiamo la stabilità rispetto alla norma per il prodotto $C = AB$ dove $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times k}$ come

$$\|\hat{C} - C\| \leq f_{\text{alg}}(n)\|A\|\|B\| + O(\varepsilon^2),$$

dove $\|\cdot\|$ è la norma del massimo, ε è la precisione di macchina, e f_{alg} è una costante polinomiale che dipende dall'algoritmo. Ad esempio, $f_{\text{alg}}(n) = n^2$ per l'algoritmo di moltiplicazione classico, senza alcuna ipotesi sull'ordinamento dei calcoli svolti dal prodotto scalare. Osserviamo che f_{alg} è indipendente dalle matrici di input, e $\|A\|\|B\|$ è indipendente dall'algoritmo.

³senza ulteriori modifiche, come manipolazione algoritmica per ridurre l'errore di arrotondamento

esperimento (se $\exists$ tempo: generalizzare CP matmul in rust per supportare l'esercizio del laboratorio. A questo punto, provare a disegnare quel plot dell'errore (magari confrontalo con l'errore di Strassen e mettilo qui))

2 APPENDICE

2.1 IL MASTER METHOD

Introduciamo brevemente un metodo per risolvere relazioni ricorsive definite per ricorrenza, detto *master method*. I lettori più interessati possono trovare una trattazione più dettagliata sul libro [10]. Il master method si usa per risolvere ricorrenze della forma

$$(2.1) \quad T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

dove $a \geq 1$ e $b > 1$ sono costanti e $f(n)$ è una funzione asintoticamente positiva. La ricorrenza 2.1 descrive il tempo di esecuzione di un algoritmo che divide un problema di dimensione n in a sottoproblemi, ognuno di dimensione $\frac{n}{b}$. Gli a sottoproblemi vengono risolti ricorsivamente in tempo $T(\frac{n}{b})$, e la funzione $f(n)$ rappresenta il costo di dividere il problema e ricombinare la ricorsione di ogni sottoproblema.

Definizione 2.1.1 (Notazione O –grande/piccolo). Per delle funzioni f, g a variabile reale (o intera) nella variabile x diciamo che:

- $f(x) = O(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $|f(x)| \leq C|g(x)|$ per ogni $x \geq x_0$.
- $f(x) = o(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|f(x)|}{|g(x)|} = 0$.
- $f(x) = \Omega(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $C|f(x)| \geq |g(x)|$ per ogni $x \geq x_0$.
- $f(x) = \omega(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|g(x)|}{|f(x)|} = 0$.
- $f(x) = \Theta(g(x))$ se $f(x) = O(g(x))$ e $f(x) = \Omega(g(x))$.

Teorema 2.1.2 (Master theorem). Siano $a \geq 1$ e $b > 1$ delle costanti, sia $f(n)$ una funzione e definiamo la legge ricorsiva $T : \mathbb{N} \rightarrow \mathbb{R}$ con la ricorrenza

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

2 Appendice

dove con $\frac{n}{b}$ indichiamo sia $\lfloor \frac{n}{b} \rfloor$ o $\lceil \frac{n}{b} \rceil$. Allora $T(n)$ assume i seguenti comportamenti asintotici:

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche $\varepsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$ allora $T(n) = \Theta(n^{\log_b a \log n})$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche costante $\varepsilon > 0$, e se $af(\frac{n}{b}) < cf(n)$ per qualche costante $c < 1$ e ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$.

Esempio 2.1.3. Nel caso dell'algoritmo naive la ricorrenza ha la forma $a = 8$, $b = 2$ e $f(n) = \Theta(n^2)$, ovvero

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2).$$

Siccome $n^{\log_b a} = n^{\log_2 8} = n^3$, che è polinomialmente più grande di $f(n)$, ovvero $f(n) = O(n^{3-\varepsilon})$ per $\varepsilon = 1$, dal caso 1 di 2.1.2 segue che $T(n) = \Theta(n^3)$. Nel caso di Strassen, si ripetono i calcoli con $a = 7$, $b = 2$ e $f(n) = \Theta(n^2)$ e si ottiene la relazione $T(n) = 7T(\frac{n}{2}) + \Theta(n^2)$ e analogamente $T(n) = \Theta(n^{\log_2 7})$.

ACRONIMI

ALS	Alternating Least Squares
APA	Arbitrary Precision Approximating
CP	Canonical Polyadic Decomposition
SVD	Singular Value Decomposition

GLOSSARIO

L ^A T _E X	A document preparation system
$\mathbb{R}$	The set of real numbers

BIBLIOGRAFIA

1. J. Alman, R. Duan, V. V. Williams, Y. Xu, Z. Xu, e R. Zhou. «More asymmetry yields faster matrix multiplication». *Computing Research Repository (arXiv)*, 2024. arXiv:2404.16349.
2. J. Alman, V. Vassilevska Williams et al. «Improving the matrix multiplication exponent with modern optimization and AlphaEvolve». *arXiv preprint*, 2026.
3. G. Ballard, A. R. Benson, A. Druinsky, B. Lipshitz, e O. Schwartz. «Improving the numerical stability of fast matrix multiplication». *SIAM Journal on Matrix Analysis and Applications* 37:4, 2016, pp. 1382–1418.
4. D. Bini. «Relations between exact and approximate bilinear algorithms. Applications». *Calcolo* 17:1, 1980, pp. 87–97. DOI: [10.1007/bf02575865](https://doi.org/10.1007/bf02575865). URL: <https://doi.org/10.1007/bf02575865>.
5. D. Bini e G. Lotti. «Stability of fast algorithms for matrix multiplication». *Numerische Mathematik* 36:1, 1980, pp. 63–72.
6. M. Bläser. «A $(5/2)n^2$ -Lower Bound for the Multiplicative Complexity of $n \times n$ -Matrix Multiplication». In: *Proceedings of the 18th Annual Symposium on Theoretical Aspects of Computer Science*. STACS '01. Springer-Verlag, Berlin, Heidelberg, 2001, pp. 99–109. ISBN: 3540416951.
7. M. Bläser. «On the complexity of the multiplication of matrices of small formats». *Journal of Complexity* 19:1, 2003, pp. 43–60. ISSN: 0885-064X. DOI: [https://doi.org/10.1016/S0885-064X\(02\)00007-9](https://doi.org/10.1016/S0885-064X(02)00007-9). URL: <https://www.sciencedirect.com/science/article/pii/S0885064X02000079>.
8. P. Bürgisser, M. Clausen, e M. A. Shokrollahi. *Algebraic complexity theory*. Vol. 315. Springer Science & Business Media, 2013.
9. D. Coppersmith e S. Winograd. «Matrix multiplication via arithmetic progressions». *Journal of Symbolic Computation* 9:3, 1990. Computational algebraic complexity editorial, pp. 251–280. ISSN: 0747-7171. DOI: [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2). URL: <https://www.sciencedirect.com/science/article/pii/S0747717108800132>.

10. T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein. *Introduction to algorithms*. MIT press, 2022.
11. P. D’Alberto. *Strassen’s Matrix Multiplication Algorithm Is Still Faster*. 2023. arXiv: [2312.12732](https://arxiv.org/abs/2312.12732) [cs.MS]. URL: <https://arxiv.org/abs/2312.12732>.
12. A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatin, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz et al. «Discovering faster matrix multiplication algorithms with reinforcement learning». *Nature* 610:7930, 2022, pp. 47–53.
13. F. L. Gall. «Powers of Tensors and Fast Matrix Multiplication». *CoRR* abs/1401.7714, 2014. arXiv: [1401.7714](http://arxiv.org/abs/1401.7714). URL: <http://arxiv.org/abs/1401.7714>.
14. J. D. Laderman. «A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications». *Bulletin of the American Mathematical Society* 82:1, 1976, pp. 126 –128.
15. J. M. Landsberg. «Geometry and complexity theory». *NSF Award Number 1405348. Directorate for Mathematical and Physical Sciences* 14:1405348, 2015, p. 5348.
16. J. M. Landsberg. *Tensors: geometry and applications*. Vol. 128. American Mathematical Soc., 2011.
17. W. Miller. «Computational complexity and numerical stability». In: *Proceedings of the sixth annual ACM symposium on Theory of computing*. 1974, pp. 317–322.
18. V. Y. Pan. «Strassen’s algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations». In: *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. 1978, pp. 166–176. DOI: [10.1109/SFCS.1978.34](https://doi.org/10.1109/SFCS.1978.34).
19. A. Schönhage. «Partial and Total Matrix Multiplication». *SIAM Journal on Computing* 10:3, 1981, pp. 434–455. DOI: [10.1137/0210032](https://doi.org/10.1137/0210032). eprint: <https://doi.org/10.1137/0210032>. URL: <https://doi.org/10.1137/0210032>.
20. A. Stothers. «On the complexity of matrix multiplication». Tesi di dott. Edinburgh, UK: University of Edinburgh, 2010. URL: <https://ed.ac.uk>.

21. V. STRASSEN. «Relative bilinear complexity and matrix multiplication.» *Journal für die reine und angewandte Mathematik* 1987:375-376, 1987, pp. 406–443. DOI: [doi:10.1515/crll.1987.375-376.406](https://doi.org/10.1515/crll.1987.375-376.406). URL: <https://doi.org/10.1515/crll.1987.375-376.406>.
22. V. Strassen. «Gaussian elimination is not optimal». *Numer. Math.* 13:4, 1969, pp. 354–356. ISSN: 0029-599X. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://doi.org/10.1007/BF02165411>.
23. V. V. Williams. *Breaking the Coppersmith-Winograd barrier*. Manuscript available at <http://theory.stanford.edu/~virgi/matrixmult-f.pdf>. 2011.
24. S. Winograd. «On multiplication of 2×2 matrices». *Linear Algebra and its Applications* 4:4, 1971, pp. 381–388. ISSN: 0024-3795. DOI: [https://doi.org/10.1016/0024-3795\(71\)90009-7](https://doi.org/10.1016/0024-3795(71)90009-7). URL: <https://www.sciencedirect.com/science/article/pii/0024379571900097>.